Assignment Introduction: Understanding Retrieval-Augmented Generation (RAG)

Large Language Models (LLMs) are powerful, but they have fundamental limitations. They can only use the knowledge stored inside their parameters, which means they:

- cannot access new or updated information
- cannot reference external documents
- may hallucinate facts that do not exist
- cannot provide grounded or verifiable sources

To address these issues, modern AI systems use a technique called Retrieval-Augmented Generation (RAG).

What is RAG?

Retrieval-Augmented Generation combines two components:

1. Retriever

The retriever searches an external knowledge base and finds the most relevant text chunks for a given query.

It requires the following steps:

- splitting documents into smaller chunks
- embedding text into numerical vectors
- computing similarity between vectors
- selecting the top-k relevant chunks

2. Generator

The generator is the LLM that uses the retrieved text to produce a more accurate and grounded answer.

By adding retrieved context to the prompt, the LLM is able to answer questions more accurately that matches the context and avoid hallucination.

Why RAG Matters

RAG significantly improves the behavior of LLMs:

- reduces hallucinations

- provides factual grounding and citations
- enables domain-specific knowledge (medical, legal, academic, etc.)
- allows the model to use new information without retraining
- improves accuracy on specialized tasks

Why This Assignment?

In this assignment, you will implement a RAG pipeline from scratch, without using FAISS, LangChain, or other retrieval libraries.

You will manually build each major component:

1. manual chunking
2. sentence-transformer embeddings
3. manual cosine similarity
4. manual retrieval
5. prompt construction
6. generation with a small HuggingFace model
7. comparison between RAG and non-RAG answers